Lab Experiment 11: Node.js Backend Development (API-First Design)

Table of Contents

Lab Experiment 11: Node.js Backend Development (API-First Design) .. 1

Lab Overview 1

Objective 1

API-First Design 2

Tasks 3

Lab Overview

In this lab, students will continue building the Task Manager backend system by creating a Node.js and Express-based REST API connected to PostgreSQL. Students will follow an API-first development approach, where API contracts and endpoint design are defined before implementation begins.

The lab introduces students to modern backend engineering concepts including modular architecture, separation of concerns, asynchronous programming, database integration, and AI-assisted backend development.

By the end of this lab, students will have implemented a functional backend API capable of performing CRUD operations on tasks stored in PostgreSQL

Objective

By completing this lab, you will learn how to:

1. Set up a Node.js and Express backend server
2. Connect Node.js applications to PostgreSQL databases
3. Build RESTful API endpoints
4. Use asynchronous programming with async/await
5. Apply API-first development principles
6. Organize backend code using separation of concerns

7. Test APIs using Postman
8. Use AI tools to assist backend development while critically reviewing generated code
9. Implement validation and structured error handling

Technologies Used

- Node.js
- Express.js
- PostgreSQL
- Pg
- Postman
- Dotenv
- VS Code

API-First Design

Modern backend systems are commonly built using an API-first approach. This means developers first define how clients will communicate with the server before implementing backend logic.
Benefits of API-first design:

- Clear communication between frontend and backend teams
- Easier testing and debugging
- Better scalability and maintainability
- More predictable system architecture
- Easier API documentation generation

In this lab, you will first define API contracts and then implement the backend routes.

API Contract (given FIRST)

- GET /tasks: Get all tasks
- POST /tasks: Create a task
- PUT /tasks/:id: Update a task
- DELETE /tasks/:id: Delete a task
- POST /auth/register: Register user
- POST /auth/login: Login user

Project Structure

```
project/
|
├── server.js
├── package.json
├── .env
|
├── db/
|   └── db.js
|
├── routes/
|   ├── taskRoutes.js
|   └── authRoutes.js
|
├── controllers/
|   ├── taskController.js
|   └── authController.js
|
├── middleware/
|   └── errorHandler.js
|
└── utils/
```

Tasks

Step 1 — Initialize Node.js Project

- Open VS Code.
- Open the integrated terminal.
- Create project folder:
  `mkdir task-manager-api`
  `cd task-manager-api`

- Initialize Node.js:
  `npm init -y`

- This creates: package.json
  `--Screenshot`

Step 2 — Install Dependencies

- Install required packages:
  `npm install express pg dotenv`

- Optional development dependency:
  `npm install --save-dev nodemon`

Package Purpose

- Express: Backend framework
- pg: PostgreSQL client
- dotenv : Loads environment variables
- nodemon: Automatically restarts server

Step 3 — Create Basic Express Server

Create:

`server.js`

Add:

```
const express = require('express');
const app = express();

app.use(express.json());

const PORT = 3000;

app.listen(PORT, () => {
console.log(`Server running on port ${PORT}`);
});
--screenshot
```

Understanding express.json()

```
app.use(express.json());
```

This middleware:

- parses incoming JSON requests
- converts JSON into JavaScript objects
- stores parsed data inside:
  `req.body`

Without this middleware, Express cannot properly read JSON request bodies.

```
--screenshot
```

Start Server

Run:

```
node server.js
```

Or with nodemon:

```
npx nodemon server.js
```

Expected output:

```
Server running on port 3000
```

```
--screenshot
```

Step 4 — Create Environment Variables

Create:

```
.env
```

Add:

```
DB_USER=postgres
DB_PASSWORD=your_password
DB_HOST=localhost
DB_PORT=5432
DB_NAME=task_manager_db
```

Why Use Environment Variables?

Environment variables help:

1. protect sensitive information
2. avoid hardcoding credentials
3. support deployment environments
4. improve security practices

Step 5 — Connect PostgreSQL

Create folder:

```
/db
```

Create:

```
db.js
```

Add:

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
user: process.env.DB_USER,
host: process.env.DB_HOST,
database: process.env.DB_NAME,
password: process.env.DB_PASSWORD,
port: process.env.DB_PORT,
});

module.exports = pool;
```

Understanding Pool

The PostgreSQL Pool:

```
const pool = new Pool()
```

creates reusable database connections.

Benefits:

- better performance
- fewer connection overheads
- scalable architecture
- production-ready design

Step 6 — Test Database Connection

Add temporary test code inside:

```
server.js
const db = require('./db/db');

db.query('SELECT NOW()', (err, result) => {
if (err) {
console.error(err);
} else {
console.log(result.rows);
}
});
```

Expected output:

```
current timestamp

-- Successful PostgreSQL connection log
```

Step 7 — Create Routes Folder

Create:

```
/routes
```

Create:

```
taskRoutes.js
```

Step 8 — Create GET /tasks Endpoint

Inside:

```
const express = require('express');
const router = express.Router();
const db = require('../db/db');

router.get('/tasks', async (req, res) => {
try {
const result = await db.query('SELECT * FROM tasks');
res.json(result.rows);
} catch (error) {
console.error(error);
res.status(500).json({ error: 'Server error' });
}
});

module.exports = router;
```

Understanding async/await

```
async (req, res)
```

allows asynchronous operations.

```
await db.query()
```

waits for the database query to finish before continuing.

Benefits:

- cleaner syntax
- easier error handling
- avoids callback nesting

Step 9 — Connect Routes to Server

Inside:

server.js

Add:

```
const taskRoutes = require('./routes/taskRoutes');
app.use('/', taskRoutes);
```

Full server.js Example

```
const express = require('express');
const app = express();
const taskRoutes = require('./routes/taskRoutes');

app.use(express.json());
app.use('/', taskRoutes);

const PORT = 3000;

app.listen(PORT, () => {
console.log(`Server running on port ${PORT}`);
});
```

Step 10 — Test API with Postman

Open Postman.

Send request:

GET http://localhost:3000/tasks

Expected result:

```
[
{
"id": 1,
"title": "Finish Lab",
"status": "todo"
}
]
```

-- Postman GET request result

Step 11 — Implement POST /tasks

Add:

```
router.post('/tasks', async (req, res) => {
try {
const { title, description, status, user_id } = req.body;

const result = await db.query(
`INSERT INTO tasks
(title, description, status, user_id)
VALUES ($1, $2, $3, $4)
RETURNING *`,
[title, description, status, user_id]
);

res.status(201).json(result.rows[0]);
} catch (error) {
console.error(error);
res.status(500).json({ error: 'Server error' });
}
});
```

Understanding Parameterized Queries

```
VALUES ($1, $2, $3, $4)
```

helps prevent:

- SQL injection attacks
- malicious user input
- unsafe query construction

Always use parameterized queries in production applications.

Step 12 — Implement PUT /tasks/:id

Add:

```
router.put('/tasks/:id', async (req, res) => {
try {
const { id } = req.params;
const { status } = req.body;
```

```
const result = await db.query(
'UPDATE tasks SET status = $1 WHERE id = $2 RETURNING *',
[status, id]
);

res.json(result.rows[0]);
} catch (error) {
console.error(error);
res.status(500).json({ error: 'Server error' });
}
});
```

Step 13 — Implement DELETE /tasks/:id

Add:

```
router.delete('/tasks/:id', async (req, res) => {
try {
const { id } = req.params;

await db.query('DELETE FROM tasks WHERE id = $1', [id]);

res.json({ message: 'Task deleted' });
} catch (error) {
console.error(error);
res.status(500).json({ error: 'Server error' });
}
});
```

Screenshot --POST request in Postman

PUT request in Postman

DELETE request in Postman

Step 14 — Create Authentication Routes

Create:

authRoutes.js

Add temporary placeholder routes:

```
const express = require('express');
const router = express.Router();

router.post('/auth/register', (req, res) => {
res.json({ message: 'Register endpoint' });
});
```

```
router.post('/auth/login', (req, res) => {
res.json({ message: 'Login endpoint' });
});

module.exports = router;
```

Why Create Placeholder Routes?

API-first design often starts with:

- endpoint definitions
- contracts
- expected request/response structure

before full implementation.

This allows frontend and backend teams to work simultaneously.

Step 15 — Add Auth Routes to Server

Inside:

```
server.js
```

Add:

```
const authRoutes = require('./routes/authRoutes');

app.use('/', authRoutes);
```

API Testing Checklist

Test the following:

- endpoint: /tasks, method: GET
- endpoint: /tasks, method: POST
- endpoint: /tasks/:id, method: PUT
- endpoint: /tasks/:id, method: DELETE
- endpoint: /auth/register, method: POST
- endpoint: /auth/login, method: POST

Common Backend Errors

- problem: cannot connect to DB, cause: PostgreSQL not running
- problem: req.body undefined, cause: Missing express.json()
- problem: Route not found, cause: Incorrect app.use()
- problem: SQL errors, cause: Incorrect table/column names

- problem: Port already in use, cause: Another server running

Debugging Tips
Use:

```
console.log()
```

Examples:

```
console.log(req.body)
console.log(req.params)
console.log(result.rows)
```

Debugging is a critical backend engineering skill.

AI-Assisted Development Activity

Modern backend developers increasingly use AI tools to accelerate implementation and debugging.
Examples:

- Gemini
- ChatGPT
- GitHub Copilot
- Claude

However, AI-generated code must always be critically reviewed.

AI Task - Prompt:

"Generate an Express.js route for CRUD operations on a PostgreSQL tasks table."
Students must:

1. Review AI code
2. Fix errors (this is critical learning)

Required AI Analysis
After generating AI output:

1. Review the generated code carefully
2. Identify mistakes or weaknesses
3. Fix incorrect logic
4. Improve security or structure if necessary
5. Compare AI output with your implementation

Bonus Question

1. Add validation middleware
2. Handle errors properly

OPTIONAL:

Bonus Challenge 1 — Validation Middleware

Create validation middleware.
Example:

```
function validateTask(req, res, next) {
const { title } = req.body;

if (!title) {
return res.status(400).json({
error: 'Title is required'
});
}

next();
}
```

Using Middleware

```
router.post('/tasks', validateTask, async (req, res) => {
```

Middleware runs before route handlers.
Benefits:

- reusable validation
- cleaner routes
- centralized logic

Bonus Challenge 2 — Centralized Error Handling

Create:

```
middleware/errorHandler.js
```

Example:

```
function errorHandler(err, req, res, next) {
console.error(err.stack);

res.status(500).json({
error: 'Internal Server Error'
});
}
```

```
module.exports = errorHandler;
```

Why Centralized Error Handling Matters
Benefits:

1. consistent API responses
2. easier debugging
3. cleaner code
4. scalable architecture
5. professional backend standards

Real-World Backend Concepts Introduced
This lab introduces foundational backend engineering concepts used in industry:

1. REST APIs
2. asynchronous programming
3. relational database integration
4. modular architecture
5. separation of concerns
6. middleware
7. API testing
8. AI-assisted development
9. backend debugging
10. production-ready coding practices

Final Deliverables

Submit:

1. Complete Node.js project folder
2. Working Express server
3. PostgreSQL integration
4. CRUD API endpoints
5. Authentication route placeholders
6. Screenshots of API testing
7. AI-generated code example
8. AI analysis and corrections
9. Bonus challenges (optional)

Submission Checklist

- Express server created

- PostgreSQL connected
- CRUD routes implemented
- API tested in Postman
- Async/await used
- AI task completed
- Validation middleware added
- Error handling added

Conclusion

In this lab, you built a Node.js and Express backend connected to PostgreSQL using modern backend engineering practices. You implemented RESTful API endpoints, organized code using separation of concerns, tested APIs using Postman, and explored AI-assisted backend development workflows.

This lab establishes the foundation for upcoming topics including authentication, security, testing, Docker, and deployment.